

Document Summarizer System Design

1 Overview and Scope

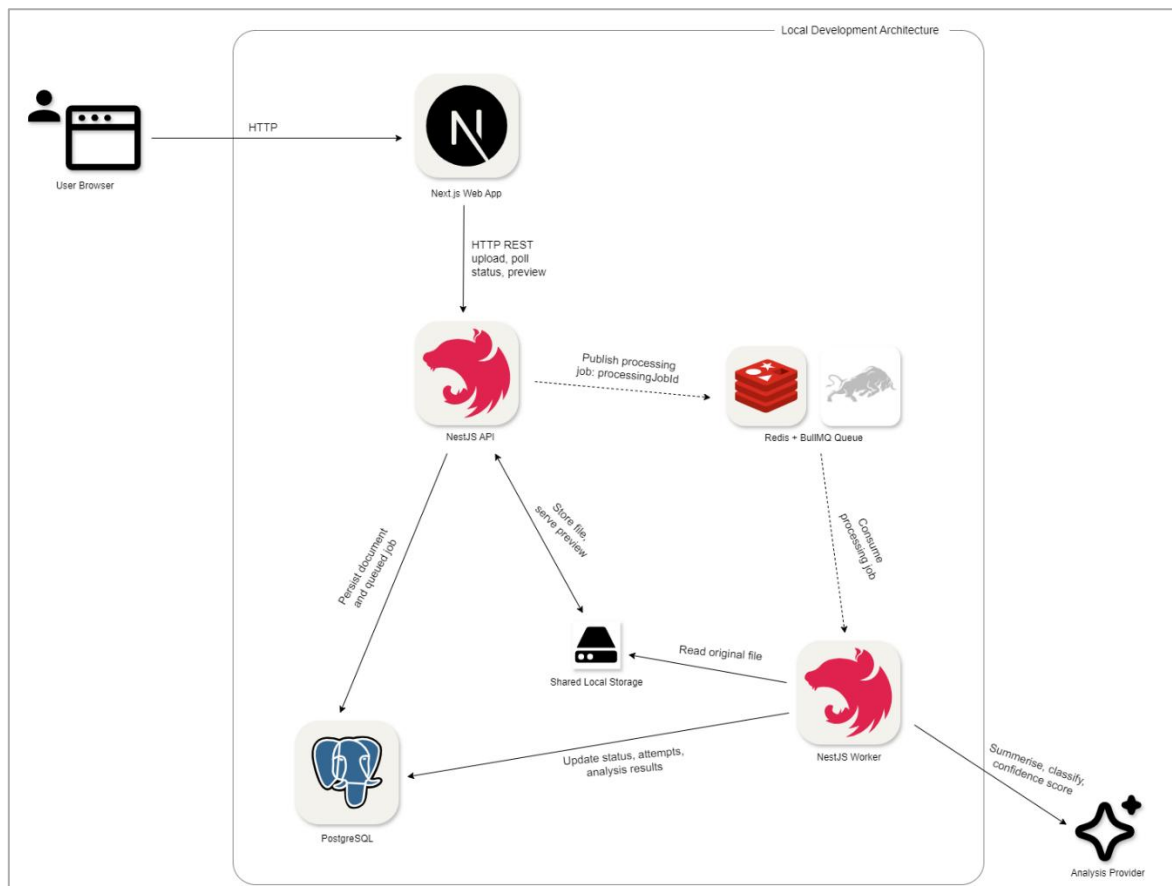
The application accepts multiple documents, processes each document asynchronously, stores the resulting summary and classification, and shows the outcome beside a preview in the browser.

Current implementation

The local scope supports TXT and PDF uploads only. Image upload and OCR are not implemented. Analysis uses MockDocumentAnalysisProvider, which produces a deterministic rule-based summary, category, and confidence score. It is not an OpenAI integration.

DocumentAnalysisProvider in `apps/worker/src/processing/document-analysis-provider.ts` is the provider abstraction for a future real analysis provider. The frontend supports multi-file upload, status polling, selected-document detail, and preview through `apps/web/src/features/documents/`.

2 Architecture and Technology Choices

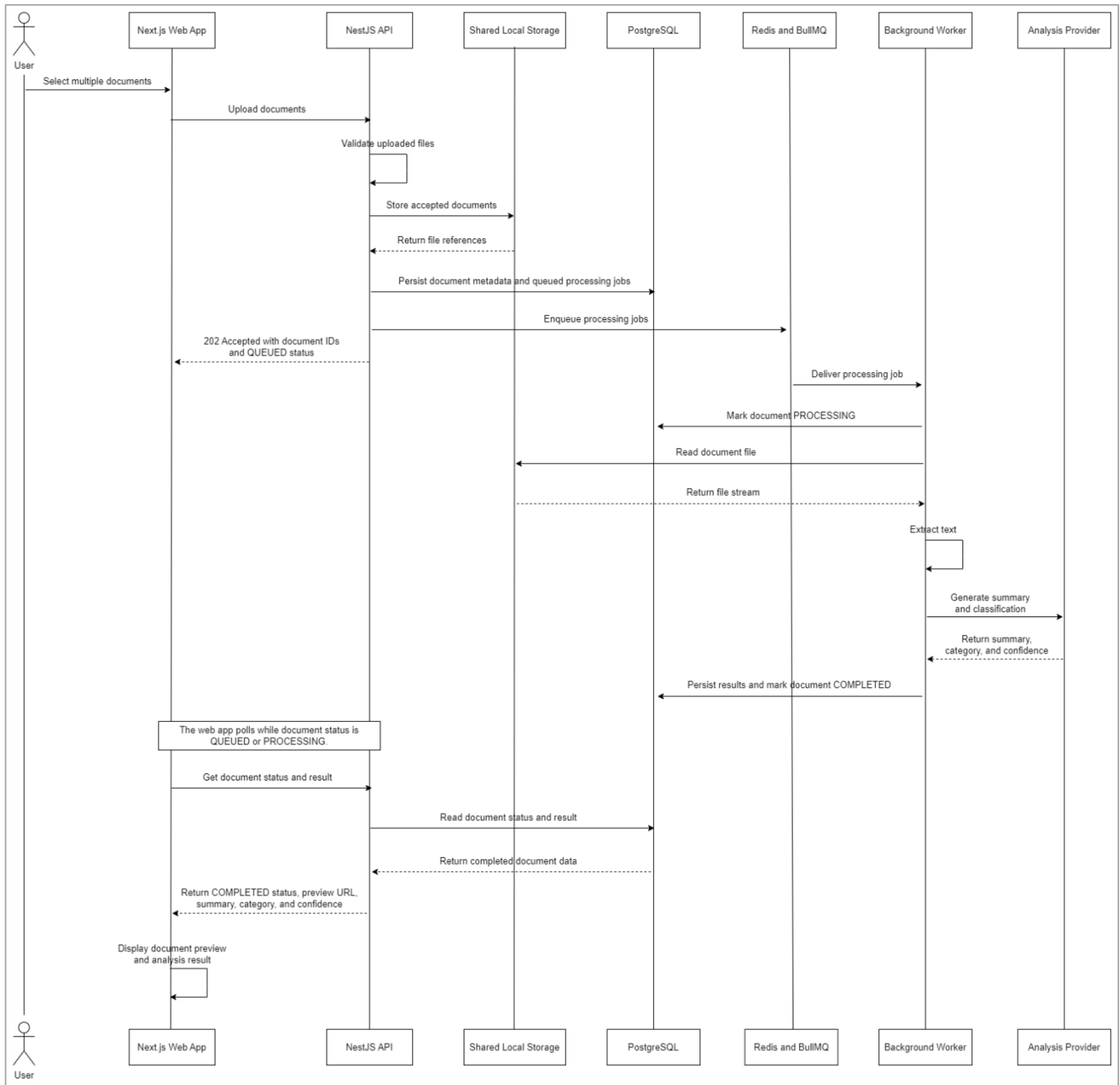


Component	Current role
Next.js web app	Browser UI for file selection, client-side validation, upload, polling, detail, and preview.
NestJS API	Receives uploads, applies server-side validation, stores files, writes durable metadata, exposes document endpoints, and publishes processing work.
PostgreSQL	Durable source of truth for documents, jobs, attempts, and analysis results. Prisma manages access through packages/database.
Redis and BullMQ	Queue backing store and delivery mechanism for document-processing jobs.
Standalone NestJS worker	Consumes BullMQ jobs, claims database work, extracts text, runs analysis, and persists terminal state. It has no HTTP server.
Local storage abstraction	DocumentStorage is implemented by LocalDocumentStorage in packages/storage; API and worker use the same configured local storage root.
Mock analysis provider	MockDocumentAnalysisProvider performs keyword-based classification and short summaries.

BullMQ keeps upload latency independent of extraction and analysis time, and provides delayed retries and worker locking. PostgreSQL keeps the durable business state separate from transient queue state. The API and worker are separate processes so HTTP capacity and processing capacity can be changed independently.

Polling is deliberately simple for the local UI: the browser polls every two seconds only while a document has active processing work. The storage abstraction keeps local filesystem storage suitable for development while preserving a replacement point for object storage later.

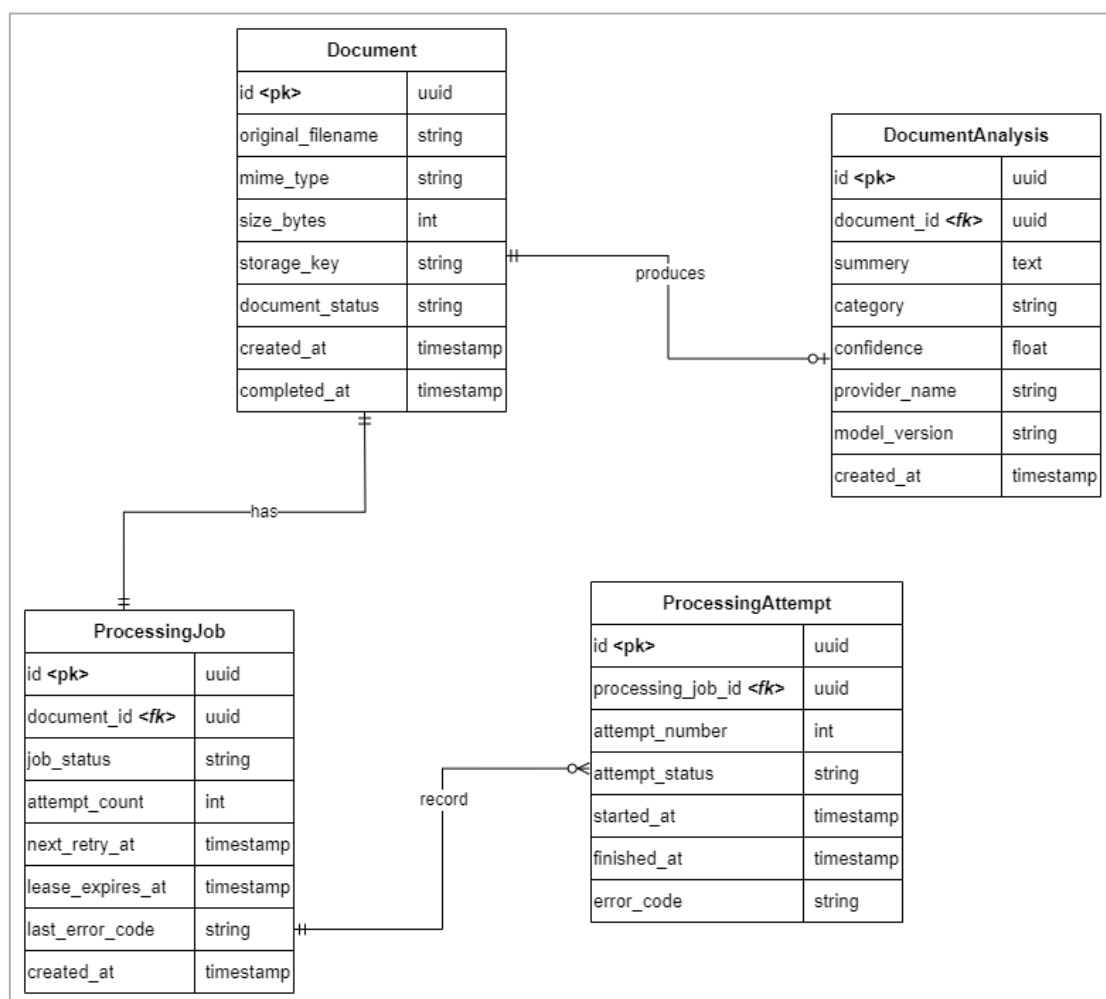
3 Upload, Processing, and Frontend Update Flow



1. The browser validates a selection of up to 10 TXT or PDF files, each at most 10 MB, and sends multipart/form-data to POST /documents.
2. The API validates the received files again. Multer stages uploads on disk, then LocalDocumentStorage streams each accepted file to its UUID storage key.
3. In one PostgreSQL transaction, the API creates one Document in QUEUED status and one ProcessingJob in PENDING status per file.

4. The API attempts to publish each processing-job ID to BullMQ. A successful publication changes the database job to QUEUED.
5. The API returns 202 Accepted with document IDs before background extraction or analysis finishes. A queue-publication failure does not turn this accepted upload into a synchronous error.
6. A worker receives the queue job, claims the matching database job, creates a ProcessingAttempt, and marks the document and job as processing.
7. The worker reads the stored file, extracts TXT or PDF text, and invokes the mock analysis provider.
8. On success, one transaction upserts DocumentAnalysis, completes the attempt and job, and marks the document COMPLETED.
9. The dashboard polls GET /documents while any job is PENDING, QUEUED, PROCESSING, or RETRY_SCHEDULED. It refreshes selected-document detail and stops polling when no active work remains.

4 Data Model



Entity	Relationship and purpose
Document	One uploaded file. It has at most one ProcessingJob and one DocumentAnalysis. Its status is QUEUED, PROCESSING, COMPLETED, or FAILED. It records storage metadata, filename, MIME type, size, and completion time.
ProcessingJob	One-to-one with Document through unique documentId. It carries queue-facing state: PENDING, QUEUED, PROCESSING, RETRY_SCHEDULED, COMPLETED, or DEAD_LETTERED; it also records retry time, lease expiry, attempt count, and last error code.
ProcessingAttempt	One-to-many from ProcessingJob. Each attempt has a unique attempt number and records STARTED, COMPLETED, or FAILED, timing, and an error code.
DocumentAnalysis	One-to-one with Document through unique documentId. It stores summary, category, confidence, provider name, and optional model version.

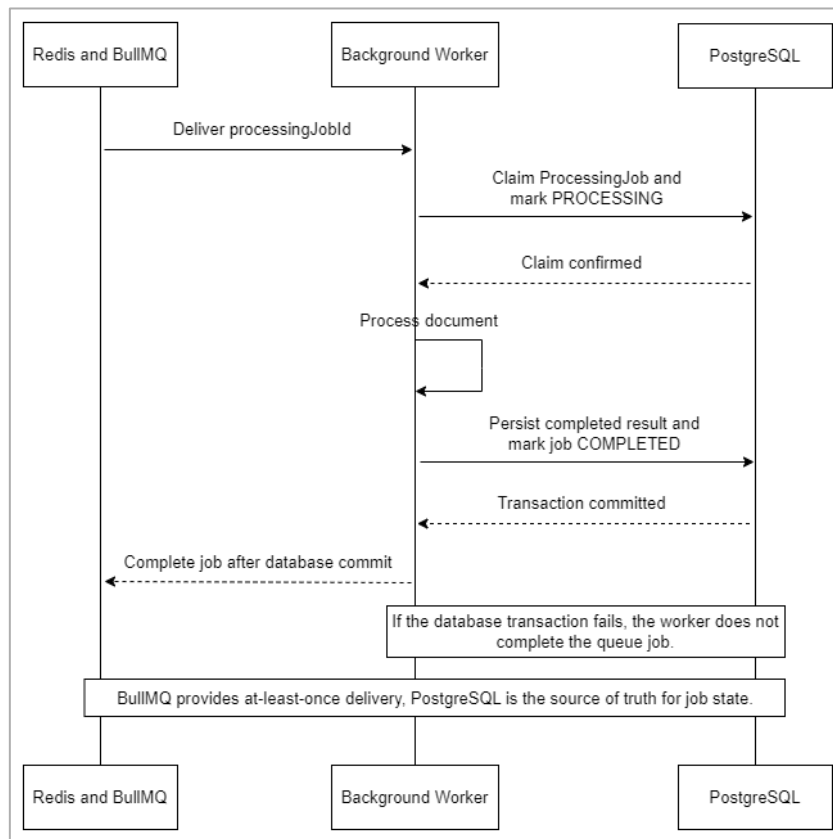
(The Prisma schema is in packages/database/prisma/schema.prisma)

The main state flow is PENDING -> QUEUED -> PROCESSING -> COMPLETED, with RETRY_SCHEDULED for retrieable failures and DEAD_LETTERED for terminal processing failures. The associated document ends in COMPLETED or FAILED.

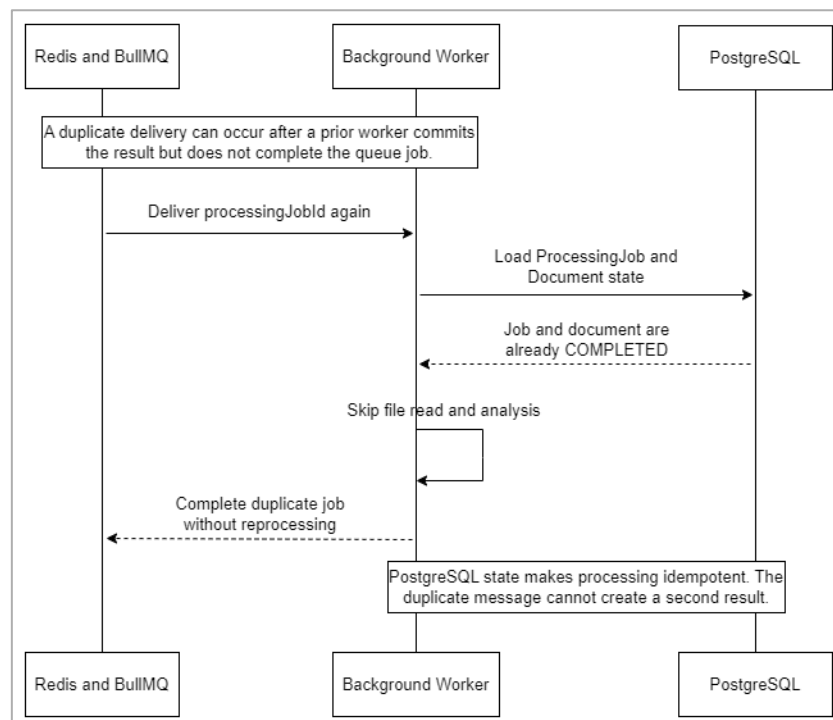
Attempts are separate from the job so the current state remains compact while each processing run retains its own timing and error outcome. This supports retry history and stale-worker recovery without overwriting earlier attempt evidence.

5 Queue Delivery and Reliability

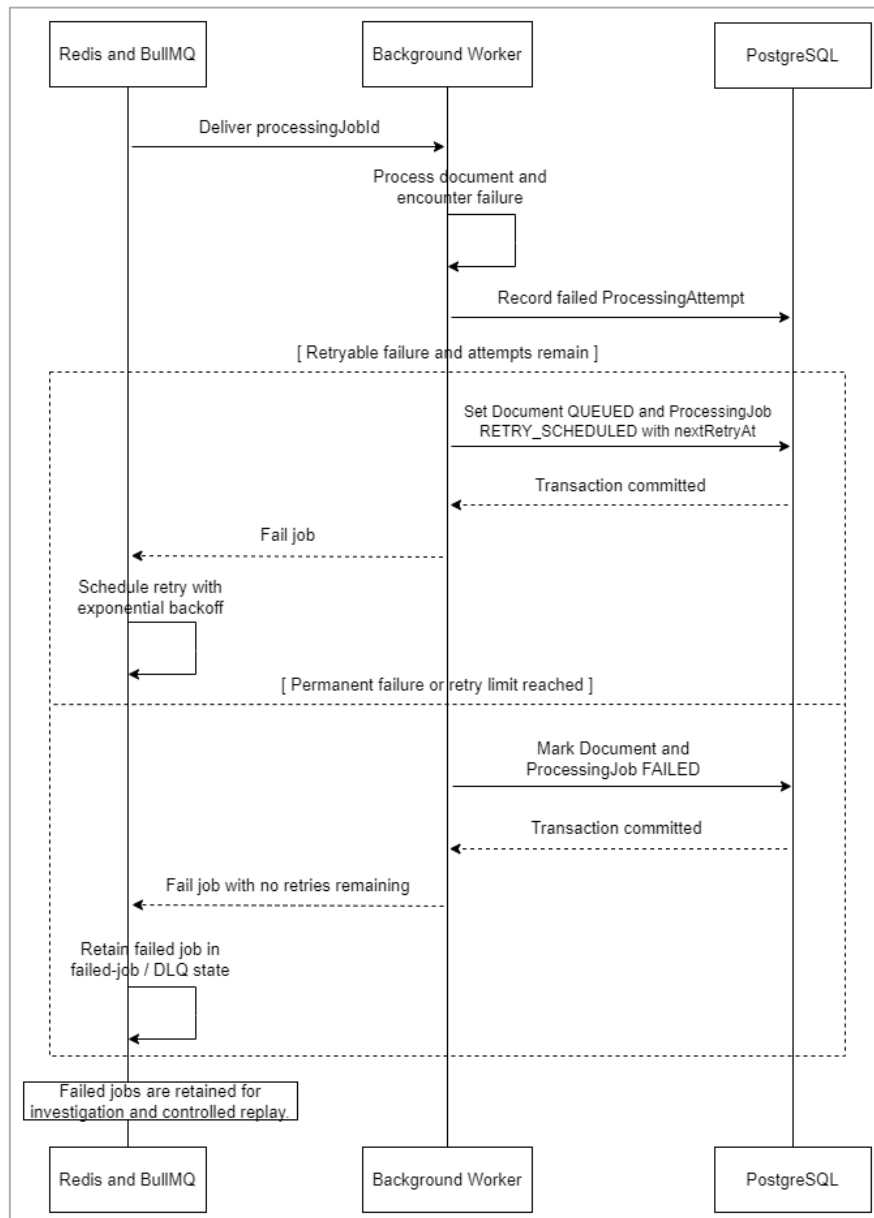
Queue delivery and acknowledgement



Duplicate delivery and idempotent processing



Retry and dead-letter flow



Current implementation

- The API stores the file and commits Document plus ProcessingJob before it attempts queue publication. This means a queue outage after the database write does not discard accepted work. PendingJobRecoveryService scans up to 100 oldest PENDING jobs on API start and every 30 seconds, then retries publication.
- BullMQ jobs use the processing-job ID as their queue job ID. They are configured for three attempts with exponential backoff beginning at one second. On a retrieable processing failure, the worker records RETRY_SCHEDULED, a nextRetryAt, and a failed attempt before rethrowing so BullMQ schedules

the retry. On the final failed attempt, the job becomes `DEAD_LETTERED` and the document becomes `FAILED` with `PROCESSING_RETRY_EXHAUSTED`.

- The normal state path is `PENDING -> QUEUED -> PROCESSING`. The worker also accepts `PENDING` when claiming an already delivered queue job. This safely handles the small race where BullMQ delivers the job before the API has completed its conditional update from `PENDING` to `QUEUED`.
- Extraction failures, unsupported MIME types, and empty extracted text are terminal errors. They are recorded as `DEAD_LETTERED` without another BullMQ retry. Dead-lettering is a database job state in this implementation, not a separate dead-letter queue.
- BullMQ delivery is at least once. The worker first claims the database job with a conditional update. A duplicate delivery whose job is already claimed or terminal returns without reading the file or creating another result. Successful processing uses a `DocumentAnalysis` upsert, which is an additional safeguard against duplicate analysis rows.
- The worker uses a 60 sec BullMQ lock and writes a matching 60 sec database lease. It resolves the queue handler only after the completion transaction commits, so BullMQ acknowledgement follows durable completion. If the completion transaction fails, the handler rejects and BullMQ does not acknowledge that delivery. A later delivery can reclaim a `PROCESSING` job only after its database lease expires, the stale attempt is marked failed and a new attempt is created, or the job is dead-lettered when the final attempt had already expired.

6 Deployment Architecture

Current local deployment

The web app, API, and worker run as separate local processes using their workspace scripts. `docker-compose.yml` starts only PostgreSQL and Redis, with persistent local Docker volumes and health checks. It does not start the web app, API, or worker.

Uploaded files are stored on the local filesystem under `STORAGE_ROOT` (default `data/uploads`). The API and worker therefore need access to the same local storage root. This is a local-development arrangement, not a full-stack Compose deployment.

Production evolution

In a production deployment, a CDN or load balancer would route users to the web application and stateless API replicas. API replicas would persist document and job state in managed PostgreSQL, publish work to Redis/BullMQ, and store uploaded files through an object-storage implementation of DocumentStorage.

Independently scalable worker replicas would consume jobs from the shared Redis/BullMQ queue, read source files from object storage, update PostgreSQL processing records, and optionally call a real analysis provider through DocumentAnalysisProvider.

Runtime secrets would be supplied through a secrets manager rather than local environment files. Managed

Managed database and queue services, replicas where appropriate, backups, restore testing, and durable object storage improve availability over the local single-instance setup. Backlog and dead-letter metrics should drive capacity decisions and alerting.

8 Failure Scenarios and Recovery

Failure scenario	Current behaviour and recovery
Queue unavailable during upload	File and DB records are persisted first. The job remains PENDING; recovery retries publication every 30 seconds and at API startup.
Worker crash	Queue delivery is not acknowledged; after lock/lease expiry, a later delivery can reclaim the job.
Duplicate delivery	Conditional database claim prevents repeated file processing and duplicate analysis.
Temporary provider failure	Attempt fails, job becomes RETRY_SCHEDULED, and BullMQ retries with exponential backoff.
Retry exhaustion	Job becomes DEAD_LETTERED; document becomes FAILED.
Malformed PDF / empty text	Terminal failure; no retry is scheduled.
Database unavailable	Upload is not accepted; worker-side stranded-job reconciliation is a production improvement.
File missing from storage	Terminal extraction failure; document is marked failed.

9 Observability and Health

Current implementation

The API exposes GET /health, which returns API and database state and reports 503 when its database query fails. Swagger is available at /api. API and worker services use NestJS logs for startup, worker, processing events, and error conditions but more structured lifecycle logging is a production improvement.

PostgreSQL job and attempt records retain processing state and error codes. The frontend makes those statuses visible through its document list and selected-document detail. The worker has no HTTP health endpoint because it is a Nest application context rather than an HTTP application.

Signal	Current local mechanism	Production evolution
API and database availability	API GET /health performs a database query.	External health checks and alerts.
API contract inspection	Swagger at /api.	Versioned API documentation and contract monitoring where needed.
Worker activity and failures	Worker logs plus ProcessingJob and ProcessingAttempt records. No worker health endpoint	Worker readiness/liveness signal, process metrics, and alerts.
Processing outcome	Database statuses and frontend status badges/detail.	Processing latency, success rate, retry, and dead-letter metrics.
Queue pressure	No metrics endpoint or dashboard.	Queue depth, age, delayed-job, and publication-recovery metrics with alerts.
Cross-service diagnosis	Logs only no tracing.	Structured logs, distributed tracing, dashboards, and alerting.

Metrics, tracing, alerting, and dashboards are production improvements, not current features.

10 Security, Privacy, and Retention

Current implementation

The API validates file count, MIME type, size, and emptiness server-side; the browser repeats those checks for faster feedback. The current accepted types are only application/pdf and text/plain.

LocalDocumentStorage uses UUID storage keys, validates their UUID format before reads or deletes, and resolves paths within the configured storage root to prevent path traversal. CORS permits only the configured WEB_ORIGIN. Runtime values are read from environment files, .env is excluded from source control and example files contain no real secrets.

Current limitations and production requirements

There is no authentication or authorization, malware scanning, encrypted object storage, or automated retention/deletion. CORS is not access control. Production requires authenticated, authorized document

access; malware scanning before processing; encryption in transit and at rest; rate limiting; controlled secret delivery; and an explicit retention policy with verified deletion of file and database data.

11 Trade-offs, Limitations, and Production Evolution

Area	Current implementation	Production evolution
Polling versus WebSockets/SSE	The frontend polls every two seconds only while work is active. This keeps the local UI and API simple.	Use SSE or WebSockets if lower-latency updates or high polling volume justify the added connection management.
Mock provider versus real AI provider	MockDocumentAnalysisProvider gives deterministic keyword-based results and avoids external credentials or cost.	Bind a real provider implementation to DocumentAnalysisProvider, with provider timeouts, rate-limit handling, cost controls, and result-quality evaluation.
TXT/PDF versus image/OCR	TXT and PDF are the only supported extraction formats. Image/OCR support is not implemented.	Add an OCR extraction path with the same terminal-error and retry policy.
Local storage versus object storage	LocalDocumentStorage is simple for one local shared storage root.	Implement DocumentStorage for durable object storage, with encryption, access controls, and lifecycle deletion.
Local Compose versus full containerized deployment	Compose currently runs only PostgreSQL and Redis. web, API, and worker start separately.	Package and deploy each application service independently, keep infrastructure lifecycle separate from application scaling.
10 MB limit and in-memory extraction	The upload limit bounds per-file staging and later Buffer.concat() extraction memory.	Use streaming extraction or stricter workload-specific limits for larger documents.

OpenAI adapter	An OpenAI adapter is intentionally deferred. No OpenAI or other integration is present.	Add it as another DocumentAnalysisProvider implementation only when credentials, safety, cost, and operational policies are defined.
Worker health and observability	The worker has logs but no HTTP health endpoint. There are no metrics, tracing, alerts, or dashboards.	Add worker health signalling and an observability stack before relying on unattended production operation.
Database-outage reconciliation	Worker retries exist, but the current code has no reconciliation scan for jobs stranded in QUEUED or PROCESSING by a database outage.	Add monitored, idempotent reconciliation and controlled requeue after database recovery.